

Tema: Análisis de redes de transporte reales – Algoritmos sobre grafos con datos de benchmark

Modalidad Trabajo: Actividad grupal hasta 3 integrantes por equipo.

Entrega: Viernes 08 de mayo de 2025 hasta las 11:59 p.m.

Medio de entrega: Enlace al repositorio GitHub publicado en Interactiva Virtual, con los nombres completos de los integrantes del equipo.

Lenguaje: C / C++ (implementación obligatoria en este lenguaje)

1. Contexto y motivación

El análisis de redes de transporte es uno de los problemas clásicos de grafos con mayor impacto real. Desde la planificación de rutas en GPS hasta la optimización logística de carga, los algoritmos que estudia este curso operan directamente sobre grafos que representan millones de intersecciones y tramos viales.

Este proyecto usa el dataset roadNet-PA (Pennsylvania Road Network), parte de la colección SNAP (Stanford Network Analysis Project) y uno de los grafos de referencia del 9th DIMACS Implementación Challenge on Shortest Paths (2006), el benchmark internacional de mayor influencia en algoritmos de camino mínimo. El dataset es de libre descarga, está bien documentado y ha sido utilizado en decenas de publicaciones de investigación, lo que permite a los estudiantes verificar sus resultados contra valores publicados y posicionarlos en un contexto de ingeniería real.

Fuente del dataset:

- **URL de descarga directa:** <https://snap.stanford.edu/data/roadNet-PA.txt.gz>
- **Pagina del dataset:** <https://snap.stanford.edu/data/roadNet-PA.html>
- **Estadísticas:** 1,088,092 nodos 3,083,796 aristas grafo no dirigido representa intersecciones viales de Pennsylvania, EE.UU.
- **Citación requerida:** Leskovec, J. & Krevl, A. (2014). SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>

El grafo es disperso (promedio de ~2.8 aristas por nodo), casi totalmente conexo (componente conexa principal con 1,087,562 nodos) y tiene un diámetro de 782 saltos, propiedades que lo hacen ideal para comparar listas vs. matrices de adyacencia, estudiar el comportamiento empírico de Dijkstra, y analizar propiedades estructurales de redes reales.

2. Objetivos

- Leer, parsear y cargar un dataset de grafos de escala real (~1M nodos, ~3M aristas) en estructuras eficientes implementadas en C/C++.
- Implementar Dijkstra con heap mínimo para resolver consultas de camino mínimo punto a punto (P2P-SSSP) sobre el grafo vial completo.
- Analizar propiedades estructurales básicas del grafo (grado, componentes conexas, diámetro aproximado) y relacionarlas con los valores publicados por SNAP.
- Comparar empíricamente los tiempos de ejecución de Dijkstra contra BFS no ponderado y discutir las diferencias con base en la complejidad teórica.
- Diseñar y ejecutar un conjunto de consultas reproducibles sobre el dataset, documentar los resultados e identificar patrones propios de redes viales reales.
- Construir un README claro que permita a cualquier persona reproducir todos los resultados desde cero.

3. Descripción del problema y tareas requeridas

El programa recibe como entrada el archivo roadNet-PA.txt (o una versión preprocesada de él) y ejecuta tres módulos independientes seleccionados por argumento de línea de comandos.

Modulo A – Carga y análisis estructural del grafo

1. Parsear el archivo roadNet-PA.txt ignorando las líneas de comentario (prefijo #). Cada línea de datos contiene dos enteros u v que representan una arista dirigida; tratar el grafo como no dirigido (insertar u->v y v->u).
2. Construir la lista de adyacencia como representación principal (std::vector<std::vector<int>>). Justificar en el informe por que la lista de adyacencia es la única representación viable para este tamaño de grafo (la matriz requeriría ~1.18 TB de memoria).
3. Reportar las siguientes estadísticas del grafo cargado y verificarlas contra los valores publicados por SNAP:

Estadística	Valor SNAP publicado	Valor obtenido
Numero de nodos	1,088,092	(calcular)
Numero de aristas (no dirigidas)	1,541,898	(calcular)
Nodos en la componente conexas principal	1,087,562	(calcular)
Grado promedio	~2.83	(calcular)
Nodo de mayor grado	(identificar)	(calcular)

4. Implementar un BFS completo desde el nodo de mayor grado para estimar el diámetro aproximado (máximo de las distancias BFS calculadas desde ese nodo). El valor publicado por SNAP es 782; comentar la diferencia si la hay.
5. Detectar y reportar el número de componentes conexas usando DFS o BFS con marcado global.

Salida: archivo results/analisis_estructural.txt con todas las estadísticas.

Modulo B – Camino mínimo punto a punto: Dijkstra sobre la red vial

El dataset original no incluye pesos en las aristas. Para simular distancias viales se asignará un peso entero aleatorio uniforme entre 1 y 10 a cada arista en el momento de la carga, usando una semilla fija (srand(42)) para garantizar reproducibilidad entre ejecuciones.

1. Implementar Dijkstra con heap mínimo (std::priority_queue de STL con pares (distancia, nodo)) sobre la lista de adyacencia ponderada. Complejidad esperada: $O((V + E) \log V)$.
2. Implementar también BFS no ponderado (ignorando los pesos) para el mismo conjunto de consultas. Complejidad: $O(V + E)$.
3. Ejecutar ambos algoritmos sobre el siguiente conjunto fijo de 10 consultas P2P (pares origen-destino), de modo que los resultados sean comparables entre todos los equipos:

Consulta	Origen	Destino
Q01	1	500000
Q02	100	1000000
Q03	50000	750000
Q04	200000	800000
Q05	300000	100000
Q06	1	1087562
Q07	500000	1
Q08	250000	600000
Q09	10000	900000
Q10	400000	150000

4. Para cada consulta reportar: distancia mínima (Dijkstra ponderado), numero de saltos (BFS), nodos explorados por Dijkstra, nodos explorados por BFS, tiempo de ejecución en ms para cada algoritmo.
5. Reconstruir y guardar el camino completo (secuencia de nodos) para las consultas Q01 y Q06 usando el arreglo prev[].
6. Manejar correctamente el caso de nodos en componentes distintas (distancia = INF, camino = no existe).

Salida: archivo results/consultas_p2p.csv con columnas consulta, origen, destino, dist_dijkstra, saltos_bfs, nodos_dijkstra, nodos_bfs, t_dijkstra_ms, t_bfs_ms.

Modulo C – Analisis comparativo y subgrafo inducido

1. A partir de los resultados del Módulo B, construir el subgrafo inducido por los nodos que aparecen en los caminos reconstruidos de Q01 y Q06 (unión de ambos caminos).
2. Sobre ese subgrafo (que será pequeño, típicamente decenas a pocos cientos de nodos), ejecutar:
 - Calculo del MST (Kruskal o Prim, a elección) con los mismos pesos asignados.

- Verificación de si el subgrafo es un DAG (usando DFS con clasificación de aristas).

3. Reportar: número de nodos y aristas del subgrafo, peso total del MST, si es DAG o no.

4. Exportar el subgrafo en formato de lista de adyacencia a `results/subgrafo_caminos.txt` para que pueda cargarse con el programa de la Evaluación 03.

Salida: archivos `results/subgrafo_caminos.txt` y `results/analisis_subgrafo.txt`.

4. Estructura del repositorio en GitHub

El nombre del repositorio debe ser `EDA_PF_ApellidoNombre1_ApellidoNombre2`.

`EDA_PF_ApellidoNombre1_ApellidoNombre2/`

— README.md	instrucciones completas de compilación y ejecución
— data/	
— roadNet-PA.txt	dataset (descargado por el estudiante; NO subir el .gz)
— results/	
— analisis_estructural.txt	estadísticas del grafo vs. valores SNAP
— consultas_p2p.csv	resultados de las 10 consultas P2P
— subgrafo_caminos.txt	subgrafo inducido por Q01 y Q06
— analisis_subgrafo.txt	MST y análisis DAG del subgrafo
— tiempos.csv	resumen consolidado de tiempos por modulo
— src/	
— graph.hpp / graph.cpp	lista de adyacencia ponderada y no ponderada
— loader.cpp	parseo del formato SNAP (ignora líneas #)
— bfs.cpp	BFS con conteo de nodos explorados
— dijkstra.cpp	Dijkstra con heap; reconstrucción de camino
— structural.cpp	estadísticas, componentes conexas, diámetro
— subgraph.cpp	subgrafo inducido, MST, DAG
— main.cpp	CLI: selección de modulo por argumento
— report/	
— Informe.pdf	

Nota sobre el dataset: el archivo `roadNet-PA.txt` pesa aproximadamente 30 MB descomprimido. Debe incluirse en el repositorio directamente (sin comprimir) para garantizar reproducibilidad. Si GitHub rechaza el archivo por tamaño, usar Git LFS e indicarlo en el README.

5. Informe (report/Informe.pdf)

El informe debe ser un documento técnico conciso (máximo 10 páginas sin contar portada) con los siguientes apartados:

1. Introducción: descripción del problema, contexto del dataset SNAP/DIMACS y relevancia practica de los algoritmos implementados.

2. Descripción del dataset: estadísticas del grafo cargado con comparación explícita contra los valores publicados por SNAP; discusión de las propiedades de redes viales reales (dispersidad, diámetro, distribución de grados).

3. Diseño e implementación: descripción de las estructuras de datos usadas; justificación de la lista de adyacencia frente a la matriz (calcular el espacio que requeriría la matriz para este grafo); pseudocódigo de Dijkstra con heap y de la estimación de diámetro por BFS.
4. Análisis de complejidad: justificación formal de $O((V + E) \log V)$ para Dijkstra y $O(V + E)$ para BFS; discusión de por qué para este grafo disperso ambos algoritmos son viables pero la matriz de adyacencia no.
5. Resultados de consultas P2P: tabla completa con los resultados de las 10 consultas; comparación de distancias Dijkstra vs. saltos BFS; análisis de los nodos explorados y los tiempos; gráfico o tabla de tendencia.
6. Análisis del subgrafo: descripción del subgrafo inducido, MST y resultado DAG; interpretación de lo que representa geoméricamente ese subgrafo en la red vial.
7. Conclusiones: que aprendieron del comportamiento de los algoritmos sobre datos reales; diferencias con grafos sintéticos pequeños; limitaciones del enfoque usado.
8. Referencias: citar el dataset SNAP, el 9th DIMACS Challenge y el material del curso.

6. Rubrica de evaluación

Criterio	Excelente 4.6 - 5.0	Bueno 4.1 - 4.5	Satisfactorio 3.6 - 4.0	Insuficiente 3.0 - 3.5	Deficiente 0.1 - 2.9	No realizado 0
Carga y correctitud estructural	Dataset cargado correctamente; estadísticas verificadas contra SNAP; componentes conexas y diámetro correctos.	Carga correcta con leves discrepancias en estadísticas; componentes correctas.	Carga con errores menores; estadísticas incompletas o sin verificación contra SNAP.	Carga funcional, pero con errores significativos en estadísticas o componentes.	Intento de carga sin resultados utilizables.	No implementado.
Dijkstra y BFS sobre dataset real	Dijkstra con heap correcto; BFS correcto; 10 consultas ejecutadas; caminos Q01 y Q06 reconstruidos; INF manejado.	Correcto con alguna consulta fallida o sin reconstrucción de camino.	Parcial; errores en distancias o sin conteo de nodos explorados.	Solo uno de los dos algoritmos funcional; resultados incompletos.	Algoritmos presentes, pero con errores sistemáticos que impiden resultados válidos.	No implementado.
Modulo C: subgrafo, MST y DAG	Subgrafo inducido correcto; MST sobre subgrafo correcto; DAG verificado;	Subgrafo correcto; MST con leve error; DAG correcto.	Subgrafo incompleto o MST incorrecto; DAG sin clasificación de aristas.	Subgrafo construido, pero MST y DAG ausentes o incorrectos.	Intento de modulo sin resultados válidos.	No implementado.

	exportación compatible con Evaluación 03.					
Eficiencia y complejidad	Dijkstra con heap real (no arreglo lineal); justifica complejidades; discute viabilidad de lista vs. matriz para este tamaño.	Heap correcto; análisis de complejidad incompleto o sin cálculo de memoria de la matriz.	Heap funcional; análisis vago o sin relación con el dataset real.	Heap implementado incorrectamente; análisis de complejidad ausente.	Dijkstra sin heap; sin análisis de complejidad.	Sin implementación ni análisis.
Calidad del código e informe	Modular, legible, compila limpio; README reproducibile; informe con todas las secciones y resultados verificables.	Legible con carencias menores; informe adecuado, pero con detalle parcial.	Estructura confusa; informe superficial o sin métricas.	Código con errores de compilación menores; informe muy incompleto.	Código no compila; informe ausente o irrelevante.	Sin código ni informe.

7. Uso ético de herramientas de Inteligencia Artificial

Esta actividad permite el uso de herramientas de IA generativa como apoyo al aprendizaje, no como reemplazo de este.

Usos permitidos (con citación obligatoria):

- Consultar conceptos teóricos sobre algoritmos de grafos, estructuras de datos o propiedades de redes reales para reforzar la comprensión propia.
- Solicitar aclaraciones sobre errores de compilación o comportamiento inesperado del código (depurar ideas, no generar la solución completa).
- Obtener retroalimentación sobre la redacción o estructura del informe.
- Explorar variantes de implementación o estrategias de parseo para comprender diferencias conceptuales.

Usos prohibidos (constituyen plagio académico):

- Solicitar a una IA que genere el código fuente de cualquiera de los módulos (total o parcialmente) y entregarlo como trabajo propio sin comprenderlo.
- Solicitar que la IA procese el dataset, ejecute las consultas o genere los archivos de resultados.

- Copiar el informe, el análisis de complejidad o la discusión de resultados directamente desde una respuesta de IA sin reescritura y comprensión propia.
- Usar la IA para eludir el aprendizaje: si un estudiante no puede explicar su propio código en una revisión oral, la entrega se considera inválida.

Obligación de citación:

Todo contenido generado o asistido por IA debe citarse explícitamente en el informe en una sección denominada "Herramientas utilizadas", indicando: herramienta usada (ej. ChatGPT, Copilot, Claude), prompts representativos utilizados, y descripción de como ese contenido fue verificado, modificado y comprendido por el equipo. La ausencia de citación cuando hay evidencia de uso de IA se penalizará como deshonestidad académica.

Consecuencias del uso indebido:

El incumplimiento de estas normas se considera violación del reglamento estudiantil de la Universidad EAFIT y puede acarrear nota cero en la evaluación, reporte académico formal y las demás consecuencias disciplinarias establecidas en el reglamento estudiantil.

8. Instrucciones de entrega

1. Crear un repositorio público en GitHub con el nombre EDA_PF_ApellidoNombre1_ApellidoNombre2.
2. El repositorio debe contener toda la estructura descrita en la Sección 4 al momento del cierre, incluyendo el dataset roadNet-PA.txt y todos los archivos de results/.
3. Subir el enlace al repositorio en el espacio habilitado en Interactiva Virtual. Junto con el enlace, indicar claramente los nombres completos de ambos integrantes del equipo.
4. La entrega se considera válida únicamente si: el enlace de GitHub es accesible, el repositorio contiene código compilable, el dataset está presente, los archivos de resultados están generados y el informe PDF está incluido.
5. **Fecha límite:** domingo 25 de mayo de 2025 a las 11:59 p.m. No se aceptan entregas tardías salvo autorización previa y escrita del profesor.

Recordatorio: asegúrese de que el repositorio sea público antes de la fecha límite. Los repositorios privados no podrán ser evaluados y se considerarán como no entregados.

9. Lineamientos técnicos

- **Parseo del dataset:** el archivo roadNet-PA.txt contiene líneas de comentario con prefijo # que deben ignorarse. Cada línea de datos tiene el formato FromNodeId\tToNodeId. Los identificadores de nodo no son necesariamente consecutivos; usar un mapa o reindexar a 0..n-1.
- **Pesos:** asignar peso = (rand() % 10) + 1 a cada arista en el momento de inserción, con srand(42) antes del parseo. Usar el mismo peso para ambas direcciones de la arista no dirigida.
- **Heap mínimo:** usar std::priority_queue<std::pair<int,int>, std::vector<std::pair<int,int>>, std::greater<>> con pares (distance, nodo).
- **Memoria:** la lista de adyacencia de este grafo ocupa aproximadamente 50-80 MB en RAM; es viable en cualquier computador moderno. No intentar construir la matriz de adyacencia (requeriría ~4.7 TB para enteros de 32 bits).
- **Medición de tiempos:** usar std::chrono::high_resolution_clock; reportar en milisegundos con dos decimales.
- **Compilación:** g++ -std=c++17 -O2 -o eda_pf src/main.cpp src/*.cpp; el flag -O2 es obligatorio para tiempos realistas en el dataset completo.
- **Robustez:** validar que los nodos de las consultas existan en el grafo; manejar el caso de nodos en componentes distintas (reportar INF y "sin camino").
- **Reindexación:** si se reindexan los nodos, documentar claramente en el README la correspondencia entre el identificador original del dataset y el índice interno usado.

10. Verificación de resultados

Los equipos pueden verificar parcialmente sus resultados usando los valores publicados por SNAP para el dataset:

Propiedad	Valor de referencia SNAP
Nodos totales	1,088,092
Aristas no dirigidas	1,541,898
Nodos en la WCC principal	1,087,562
Coefficiente de clustering promedio	0.0465
Numero de triángulos	67,150
Diámetro (camino mas largo)	782
Diámetro efectivo al 90-percentil	530

Los resultados de las 10 consultas P2P (Modulo B) no tienen un valor de referencia único publicado porque dependen de los pesos asignados aleatoriamente con srand(42). Sin embargo, todos los equipos deben obtener exactamente los mismos valores para esas consultas si siguen el esquema de asignación de pesos indicado, lo que permite la verificación cruzada entre equipos.

11. Conexión con contenidos del curso

Este proyecto integra todos los temas de grafos trabajados durante el semestre:

Contenido del curso	Aplicación en el proyecto
Representaciones (lista vs. matriz)	Justificación obligatoria de por qué la matriz es inviable para este grafo
BFS y recorridos	Modulo A (diámetro, componentes) y Modulo B (comparación con Dijkstra)
DFS y clasificación de aristas	Modulo C (verificación DAG del subgrafo)
Dijkstra y SSSP ponderado	Modulo B (10 consultas P2P sobre ~1M nodos)
MST (Kruskal o Prim)	Modulo C (MST del subgrafo inducido)
Análisis de complejidad	Informe: $O((V+E) \log V)$ en un grafo real con $V = 1,088,092$